... notice that we do the `...classList` array unpacking in the `html_classes()` call after normal arguments, just like regular PHP. `html_classes()` takes any number of string arguments and glues them together into a single attribute string; it will also accept a mapping where key names will be included as HTML classes if their values are `true`. The final result is as follows:

```
curl http://localhost:8080/index.php
<div class="focused flex centered">
    Some content goes here.
</div>
```

Defining Functions

Twig also provides limited support for defining your own functions. The => arrow function operator behaves like its PHP counterpart: you are permitted a single (Twig) statement, which is automatically evaluated and returned. This deliberately limits what you can achieve with them, for example, you can't put Twig tags into an arrow function.

Using the {% set %} tag, you can assign the arrow function to a template variable. If the arrow function has more than one parameters; make sure to wrap them in parentheses. With that done, the template variable is then used with the |invoke filter; any arguments are passed in parentheses. Be warned that the output of an invoked arrow function will be subject to autoescaping (as it can be the recipient of non-template string data), even if that markup exists in a string literal...

```
{# views/arrow-func.html.twig #}
{% set myFunc = (arg1, arg2) => "<li>cat: #{arg1}</li>\n<li>human:
#{arg2}</li>" %}
<ul>
    {{ myFunc|invoke(cat, human) }}
</ul>
<ul>
    {{ myFunc|invoke(cat, human)|raw }}
</ul>
```